

Parallel Shark Smell Optimization

Pietro Cipriani

000000

pietro.cipriani@studenti.unitn.it

Filippo De Grandi

257824

filippo.degrandi@studenti.unitn.it

Giacomo Vettore

000000

giacomo.vettore@studenti.unitn.it

Abstract—The Shark Smell Optimization (SSO) algorithm is a metaheuristic optimization technique inspired by the hunting behavior of sharks. Due to the high computational cost of population-based optimization, this paper presents various parallel implementations of SSO using MPI, OpenMP and hybrid MPI+OpenMP programming models. The MPI version distributes the population across processes, while the hybrid approach combines distributed-memory and shared-memory parallelism to improve resource utilization on multicore systems. Performance evaluation on standard benchmark functions analyzes execution time, speedup, and scalability for different population sizes and processor configurations.

> Qui scriviamo riassunto sui risultati

Index Terms—parallelization, hpc, mpi, openmp, sharks, smelling

I. INTRODUCTION

Meta-heuristic optimization algorithms have become a widespread tool for solving complex nonlinear optimization problems where deterministic or gradient-based approaches are computationally infeasible¹. Population-based optimization methods are effective in exploring high-dimensional search spaces and handling multimodal objective functions. These techniques are widely applied in engineering optimization, machine learning, scheduling, computational biology, and scientific computing.

Although the Shark Smell Optimization (SSO) algorithm demonstrates strong optimization capabilities on nonlinear benchmark functions, its iterative population-based structure introduces substantial computational costs as the number of sharks, problem dimensionality, and iteration count increase. Parallelization becomes essential to reduce execution time and improve scalability.

This paper investigates multiple parallel implementations of the Shark Smell Optimization algorithm using Message Passing Interface (MPI), OpenMP, and hybrid MPI+OpenMP programming models. In addition to comparing distributed-memory and shared-memory paradigms, the study analyzes different parallelization strategies within the algorithm itself. Specifically, parallel execution is applied at three distinct levels:

- *shark-level* parallelism, where candidate solutions are distributed across population elements;
- *dimension-level* parallelism, where computations across search-space dimensions are parallelized;

- *hybrid* parallelism, combining both the other strategies.

By combining these algorithmic decomposition strategies with MPI, OpenMP, and hybrid MPI+OpenMP execution models, the work evaluates multiple parallel variants of SSO and analyzes their scalability, synchronization behavior, communication overhead, and computational efficiency on multicore HPC architectures.

A. The Shark Smell Optimization Algorithm

The SSO [2] algorithm is a population-based metaheuristic inspired by sharks' ability to locate prey by sensing and following odor gradients.

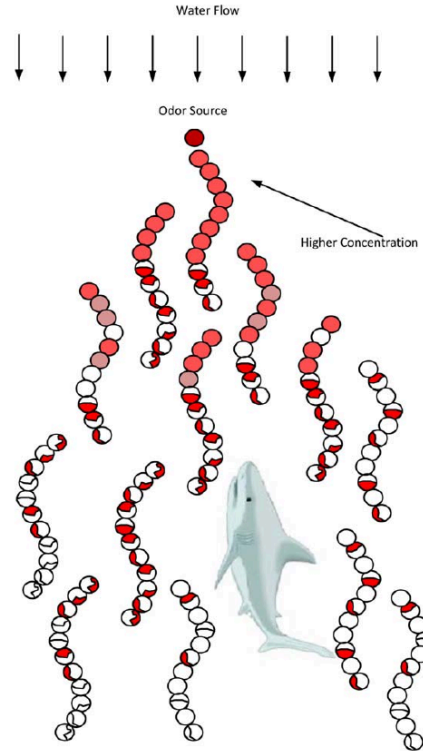


Fig. 1. Schematic illustration of a shark's movement to odor source.

The method maintains a population of candidate solutions (sharks) that move through the search space combining global exploration and focused local exploitation.

¹We found this “bestiary” with a particular hot-take on meta-heuristic algorithms [1]

```

1 struct Shark {
2     double *position; // Current position of the shark.
3     double *speed;    // Current speed of the shark.
4     double pos_score; // The OF(position) value.
5 };

```

Listing 1. Shark data structure

Each shark iteratively updates its position and speed based on a combination of inertia (momentum from previous movement) and attraction toward promising regions of the search space (global best or local neighborhood best). Additionally, a local rotational search is performed around improved sharks to refine their positions.

The main stages of the SSO algorithm are described in the following pseudo-code, which highlights the key computational steps:

```

Serial Algorithm
1 // Shark loop
2 for shark_index in 0 .. cfg.np-1:
3     shark = sharks[shark_index]
4     // Stages loop
5     for k in 0 .. cfg.k_max-1:
6         R1 = uniform(0,1)
7         R2 = uniform(0,1)
8
9         // 1) per-dimension derivative and speed update
10        for dim in 0 .. cfg.nd-1:
11            derivative = eval_derivative(shark.position,
12                                         cfg.nd, cfg.obj, dim)
13            grad_term = cfg.eta * R1 * derivative
14            mom_term = cfg.alpha * R2 * shark.speed[dim]
15            shark.speed[dim] = grad_term + mom_term
16            // apply velocity limiter using cfg.beta
17            if abs(shark.speed[dim]) > abs(cfg.beta *
18                shark.speed[dim]):
19                shark.speed[dim] = cfg.beta * shark.speed[dim]
20
21        // 2) forward movement and clamp
22        for dim in 0 .. cfg.nd-1:
23            shark.position[dim] = clamp(shark.position[dim]
24                                       + shark.speed[dim], domain[dim])
25
26        // 3) rotational local search
27        best = OF(shark.position, cfg.nd, cfg.obj)
28        best_r3 = 0
29        for m in 0 .. cfg.rotations-1:
30            r3 = uniform(-1,1)
31            for dim in 0 .. cfg.nd-1:
32                candidate[dim] = clamp(shark.position[dim] *
33                                       (1 + r3), domain[dim])
34                val = OF(candidate, cfg.nd, cfg.obj)
35                if val > best: best = val; best_r3 = r3
36            if best_r3 ≠ 0:
37                for dim in 0 .. cfg.nd-1:
38                    shark.position[dim] =
39                        clamp(shark.position[dim] * (1 + best_r3),
40                            domain[dim])

```

```

36 // 4) update global best
37 cur_min = -best
38 if cur_min < best_min:
39     best_min = cur_min
40 copy(best_pos, shark.position)
41
42 // result in best_min, best_pos

```

a) *Design intuition:*

- Inertia preserves momentum and enables smooth exploration of the search space.
- Attraction terms bias motion toward high-quality regions (global best or local neighborhood best).
- Rotational local search acts as an exploitation operator similar to a focused local search or mutation that samples around a candidate on a small hypervolume to refine its position.

II. PARALLEL DESIGN

The SSO algorithm contains various sources of parallelism. The evaluation and update of individual sharks can generally be executed independently, making shark-level decomposition highly suitable for distributed-memory execution. Conversely, dimension-level decomposition is a more complicated form of parallelism that can be applied within the evaluation of a single shark's position and speed updates, which may be more efficient in shared-memory environments. Hybrid approaches can combine these strategies to leverage both inter-node and intra-node parallelism.

In the following graph, we illustrate the slowdown of the serial algorithm as various parameters (number of sharks, dimensions, iterations) increase.

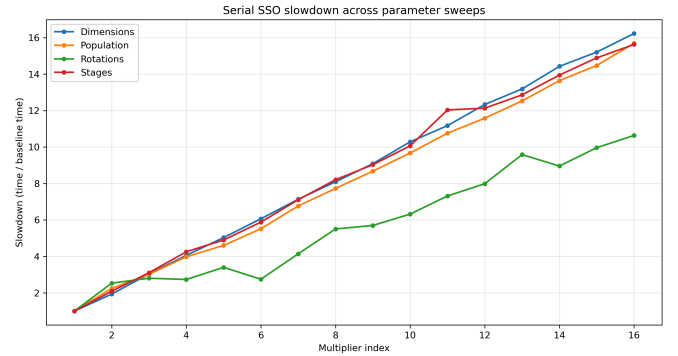


Fig. 2. Execution time of the serial SSO algorithm as a function of the number of sharks, dimensions, stages and rotations.

The graph above underlines how the SSO algorithm would benefit from parallelization, especially as the problem size increases. The computational cost grows significantly with the number of sharks, dimensions, and iterations, making it crucial to explore parallel design strategies to achieve practical execution times for larger problem instances.

Although the outer “stage” loop over the stage index k , in the graph indicated by the red line, might at first seem parallelizable, it is inherently sequential: each iteration at stage k consumes and modifies data produced at stage $k-1$.

(for example updated shark positions, velocities and fitness values). These true data dependencies prevent concurrent execution of different k iterations without violating correctness. Any approach to overlap would require complex checkpointing or synchronization that typically outweighs potential gains. Therefore, parallelism is applied at the shark and dimension levels rather than across stage iterations.

This work therefore investigates the interaction between:

- the programming model:
 - MPI,
 - OpenMP,
 - hybrid MPI+OpenMP;
- and the algorithmic decomposition strategy:
 - shark-level parallelization,
 - dimension-level parallelization,
 - rotation-level parallelization,
 - hybrid parallelization.

The comparative evaluation of these configurations enables a detailed analysis of scalability, workload distribution, communication overhead, synchronization costs, and overall parallel efficiency.

A. Shark-level Parallelism

B. Dimension-level Parallelism

C. Rotation-level Parallelism

III. IMPLEMENTATION

All code and scripts related to the project are available in the repository (see README). The implementation reuses a single algorithmic core, found in `sso/sso.c` and exposes multiple execution strategies through self-contained entrypoints in the `src` directory, e.g. `src/main_mpi_sharks.c`, `src/main_openmp_dim.c`, `src/main_hybrid_dim.c`, etc. Each entrypoint implements a specific parallelization strategy and programming model, while sharing the same underlying SSO logic.

The names of the entrypoints follow the convention `main_<model>_<strategy>.c`, where `<model>` is one of `mpi`, `openmp`, or `hybrid`, and `<strategy>` indicates the parallelization approach, i.e. `sharks`, `dim`, `rot`.

The following subsections describe the main parallelization strategies we implemented, the key code locations, and the design trade-offs.

A. MPI Implementations

We implemented three MPI decompositions that map naturally to the SSO algorithm:

- **Shark-level (population) decomposition:** each MPI rank owns a subset of the population and executes the full per-shark stages locally. At the end of the run ranks perform a reduction to determine the globally best solution. See `src/main_mpi_sharks.c`. This approach is straightforward and minimizes communication during the main loop, as each rank can independently update its sharks. This results in low communication overhead and good scalability when the population size is large

enough to amortize the cost of setup and evaluation. However, it may suffer from load imbalance if the sharks are not uniformly distributed in terms of computational cost (e.g. some sharks may converge faster than others).

- **Dimension-level decomposition:** the decision-variable vector is partitioned across MPI ranks; each rank computes updates for its set of dimensions and `MPI_Allgatherv` is used to reconstruct full position vectors when needed. See `src/main_mpi_dim.c`. Here, the main loop is parallelized across dimensions, which can be beneficial when the cost of evaluating the objective function or computing gradients is dominated by per-dimension work (e.g. very high `nd`). However, this approach introduces higher communication overhead due to the need for frequent all-gather operations to share updated positions across ranks, and it requires more complex synchronization to ensure consistency of the shared state.
- **Rotation-level decomposition:** the inner rotational local-search probes are distributed across ranks and the best candidate is found with a reduction (e.g. `MPI_MAXLOC` or custom two-value reduction). See `src/main_mpi_rot.c`. Like the dimension-level approach, this strategy can be effective when the cost of the rotational search is significant (e.g. large rotations), but it also introduces communication overhead due to the need for reductions to find the best candidate across ranks. However, it minimizes redundant objective evaluations since each rank only evaluates a subset of the rotations, and the reduction combines results efficiently.

For MPI variants we chose straightforward collective patterns (`Allreduce` / `Allgatherv` / `MPI_Bcast`) to keep the code readable and portable;

B. OpenMP Implementations

OpenMP variants exploit shared memory to avoid message passing; the code, like the MPI variants, uses different decomposition strategies depending on which loop is most expensive.

- **Shark-level (thread over sharks):** each OpenMP thread processes different sharks end-to-end and maintains thread-local scratch buffers and per-thread best values to reduce contention. See `src/main_openmp_sharks.c`. A parallel region is created through `#pragma omp parallel` and the outer loop over sharks is partitioned with `#pragma omp for`. Each thread maintains its own RNG state and scratch arrays to avoid data races and false sharing. At the end of the loop, a reduction is performed to find the global best solution across threads.
- **Dimension-level (thread over dimensions):**
- **Rotation-level (parallel rotation probes):**

OpenMP versions emphasize minimizing synchronization points and using per-thread storage for temporary arrays.

C. Hybrid MPI+OpenMP Implementations

D. Notes and Supporting Scripts

The algorithmic core (`sso_update_speed`, `sso_move_forward`, `sso_unrotational_search`, `sso_sharks_alloc`) is implemented in `src/sso/sso.c` and reused across every main entrypoint; this keeps the implementations comparable and reduces duplication.

Argument parsing and default parameters live in `src/sso/parse_args.c` and are extended in each `main_*` file to add parallel-specific options (threads, dim-procs, tsharks, etc.). Benchmarking and plotting tools are under `benchmarks/` and `tests/` (examples: `tests/launch_tests.sh`, `benchmarks/plot_results.py`). These scripts were used to generate the figures reported in Section 4.

IV. PERFORMANCE AND SCALABILITY ANALYSIS

In this section, we present a performance analysis of the various parallel implementations of the SSO algorithm. We evaluate execution time, speedup and efficiency across the different parallelization strategies and programming models. The experiments are conducted on a multicore HPC cluster, and we analyze both strong and weak scalability. The section is organized as follows:

- Experimental setup and benchmark functions
- Analysis with varying number of processes
- Analysis with varying number of threads
- Analysis with both processes and threads (hybrid)
- Analysis with different execution modes in the PBS system.

A. Experimental Setup and Benchmark Functions

To standardize our performance evaluation, we decided to stick with a fixed set of parameters for the SSO algorithm across all implementations. The set has been chosen to be sufficiently large to allow for meaningful performance measurements while keeping execution times manageable. The parameters are as follows:

- Number of sharks (`np`): 1000
- Number of dimensions (`nd`): 200
- Number of stages (`k_max`): 1000
- Number of rotations (`rotations`): 50

The rest of the parameters (such as `eta`, `alpha`, `beta`) are set to their default values as defined in the code, that have been taken from the original SSO paper.

In order to launch several tests with different parameters, we developed a set of scripts that automate the execution of the various implementations on the HPC cluster. The scripts allow us to easily vary the number of chunks to select on the cluster, processes and threads used in the MPI and OpenMP methods. Regarding the execution modes in the PBS system, we decided to stick with the `excl` mode, which ensures that the allocated resources are not shared with other jobs. This choice allows us to minimize interference and obtain more consistent performance measurements.

```
1  ...
2  for procs in "${PROC_VALUES[@]"; do
3      for thrds in "${THRD_VALUES[@]"; do
4          while [[ $(qstat -u $USER | wc -l) -ge 30 ]]; do
5              sleep 10
6          done
7          sed \
8              -e "s/\${N_THRD}/${thrds}/g" \
9              -e "s/\${N_PROC}/${procs}/g" \
10             -e "s/\${JOB_NAME}/${JOB_NAME}/g" \
11             -e "s/\${N_CPUS}/${N_CPUS}/g" \
12             -e "s/\${PLACE}/${PLACE}/g" \
13             "$EXEC" | qsub
14      done
15  done
```

Listing 2. A simplified portion of the script used to launch the tests on the HPC cluster. The script iterates over different values of processes and threads (user-defined), checks for the number of running jobs, and submits new jobs using `qsub` with the appropriate parameters.

```
1  #!/bin/env bash
2  #PBS -l
3  select=${N_CPUS}:ncpus=${N_PROC}:mpiprocs=${N_PROC}:mem
4  #PBS -l place=${PLACE}:excl
5  #PBS -l walltime=0:5:00
6  #PBS -q shortCPUQ
7  # output & error files
8  #PBS -N ${JOB_NAME}_${N_PROC}_${N_THRD}
9
10 module load OpenMPI/4.1.6-GCC-13.2.0
11
12 mpirun -n (($ {N_CPUS} * ${N_PROC} )) ./parallel-sso/
    sso_hybrid_sharks -p 1000 -d 200 -k 1000 -m 50 -t
    ${N_THRD}
```

Listing 3. An example PBS script, edited by Listing 2. The script specifies resource requirements, loads necessary modules, and runs the executable with the defined parameters for sharks, dimensions, stages, rotations, and threads.

The implementation contains three different objective functions (in `sso/ofuncs.c`), taken from the original SSO paper, that are used for testing the algorithm's performance and scalability. In our experiments, we used the Rastrigin function [3] (set as default in the code), as the differences between the various objective functions were negligible in terms of execution time and speedup.

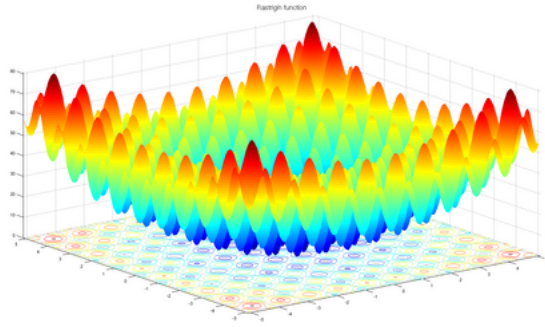


Fig. 3. Rastrigin function, a common benchmark for optimization algorithms.

B. Varying number of processes

C. Varying number of threads

D. Hybrid parallelism

E. Playing with PBS

V. CONCLUSION

This paper presented a comprehensive study of parallel implementations of the Shark Smell Optimization algorithm using MPI, OpenMP, and hybrid MPI+OpenMP programming models. By exploring different levels of parallelism (shark-level, dimension-level, rotation-level and hybrid), we analyzed the trade-offs between computational efficiency, communication overhead, and synchronization costs.

Performance evaluation on standard benchmark functions demonstrated significant reductions in execution time and improved scalability with increasing problem size and processor count. Future work could explore more advanced parallelization strategies, such as dynamic load balancing or GPU acceleration, to further enhance the performance of SSO on large-scale optimization problems.

REFERENCES

- [1] A. Campelo, "EC Bestiary: A bestiary of evolutionary, swarm and other metaphor-based algorithms," *Zenovo*, [Online]. Available: <https://fcampelo.github.io/EC-Bestiary/>
- [2] O. Abedinia, N. Amjady, and A. Ghasemi, "A new metaheuristic algorithm based on shark smell optimization," *Complexity*, vol. 21, no. 5, pp. 97–116, 2016, doi: <https://doi.org/10.1002/cplx.21634>.
- [3] Wikipedia, "Rastrigin function," [Online]. Available: https://en.wikipedia.org/wiki/Rastrigin_function